

Espressioni regolari

Espressioni regolari

Introduzione alle espressioni regolari con il modulo `re` in Python.

Perché ti servono le espressioni regolari

Immagina di dover controllare se un indirizzo email è scritto correttamente, o di dover trovare tutti i numeri di telefono in un testo. Potresti farlo con decine di condizioni `if`, oppure usare uno strumento molto più potente: le **espressioni regolari**.

Cosa sono le espressioni regolari?

Le **espressioni regolari** (dette anche "regex" o "regexp") sono una mini-lingua per descrivere **schemi di testo**. Servono a:

- Verificare se una stringa corrisponde a un certo formato ("è un'email valida?")
- Trovare schemi in un testo ("tutti i numeri in questo paragrafo")
- Estrarre informazioni ("dammi solo la data da questa riga")
- Sostituire parti di testo ("sostituisci tutti i numeri con `X`")

In Python si usano con il modulo `re` :

I mattoncini di base: cosa significano i simboli

Simbolo	Significato	Esempio
<code>.</code>	Qualsiasi carattere (eccetto a capo)	<code>a.c</code> trova "abc", "aXc", "a3c"...
<code>\d</code>	Una cifra (0-9)	<code>\d\d</code> trova "42", "07"...
<code>\D</code>	Non una cifra	
<code>\w</code>	Lettera, cifra o underscore	
<code>\s</code>	Spazio, tab o a capo	
<code>^</code>	Inizio della stringa	<code>^Ciao</code> trova solo stringhe che iniziano con "Ciao"
<code>\$</code>	Fine della stringa	<code>fine\$</code> trova solo stringhe che terminano con "fine"
<code>[abc]</code>	Uno tra a, b, c	
<code>[a-z]</code>	Una lettera minuscola qualsiasi	
<code>[0-9]</code>	Una cifra qualsiasi (come <code>\d</code>)	

Quantificatori: quante volte?

Simbolo	Significato
<code>*</code>	Zero o più volte
<code>+</code>	Una o più volte
<code>?</code>	Zero o una volta
<code>{3}</code>	Esattamente 3 volte
<code>{2,5}</code>	Da 2 a 5 volte

Cercare uno schema in una stringa

`re.search()` — cerca ovunque nella stringa

```
testo = "Il mio numero è 334-1234567"

# Schema: 3 cifre, un trattino, 7 cifre
risultato = re.search(r"\d{3}-\d{7}", testo)

if risultato:
    print(f"Numero trovato: {risultato.group()}") # → 334-1234567
else:
    print("Nessun numero trovato")
```

Il prefisso `r` prima delle virgolette (`r"..."`) indica una **raw string** (stringa "grezza"). Senza la `r`, Python interpreterebbe `\d` come una sequenza speciale (come `\n` per andare a capo), stravolgendo il significato della regex. Con la `r`, Python lascia il testo esattamente come l'hai scritto, senza trasformare niente. Usa sempre `r"..."` quando scrivi espressioni regolari.

`re.match()` — cerca solo all'inizio della stringa

```
testo = "Ciao, mondo!"
risultato = re.match(r"Ciao", testo) # La stringa inizia con "Ciao"?

if risultato:
    print("Trovato!")
else:
    print("Non trovato.")
```

`re.findall()` — trova TUTTE le corrispondenze

```
testo = "Ho 3 mele, 5 banane e 2 arance."
numeri = re.findall(r"\d+", testo)  # Trova tutti i gruppi di cifre
print(numeri)  # ['3', '5', '2']
```

Sostituire testo: `re.sub()`

```
testo = "Il prezzo è 10 euro, poi 20 euro."
nuovo = re.sub(r"\d+", "X", testo)  # Sostituisce tutti i numeri con "X"
print(nuovo)  # Il prezzo è X euro, poi X euro.
```

Dividere testo: `re.split()`

Come `str.split()`, ma puoi dividere su uno schema complesso invece che su un singolo carattere:

```
testo = "mela; banana, uva  ciliegia"
# Divide su punto e virgola, virgola o spazi (anche multipli)
parti = re.split(r"[;,\s]+", testo)
print(parti)  # ['mela', 'banana', 'uva', 'ciliegia']
```

Estrarre parti specifiche: i gruppi con `()`

Le parentesi tonde creano dei **gruppi** — puoi estrarre separatamente ogni parte del risultato:

```
testo = "Data di nascita: 15/03/2008"
match = re.search(r"(\d{2})/(\d{2})/(\d{4})", testo)

if match:
    giorno = match.group(1)    # Primo gruppo → 15
    mese = match.group(2)      # Secondo gruppo → 03
    anno = match.group(3)      # Terzo gruppo → 2008
    print(f"Giorno: {giorno}, Mese: {mese}, Anno: {anno}")
```

Riutilizzare uno schema: `re.compile()`

Se usi lo stesso schema molte volte, "compilalo" una volta sola per rendere il codice più efficiente:

```
schema_numeri = re.compile(r"\d+")    # Compila lo schema una volta

frasi = ["Ho 3 mele", "Ho 10 banane", "nessun numero qui"]
for frase in frasi:
    trovati = schema_numeri.findall(frase)
    if trovati:
        print(f"Trovati: {trovati}")
```

Esempi pratici

Verificare se un'email è valida

```
def valida_email(email):  
    # Schema: caratteri@caratteri.dominio (almeno 2 lettere)  
    schema = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"  
    return bool(re.match(schema, email))  
  
print(valida_email("alice@esempio.it"))    # True ← valida  
print(valida_email("non_valida.com"))      # False ← manca la @  
print(valida_email("@dominio.it"))         # False ← manca la parte prima  
della @
```

Verificare un numero di telefono italiano

```
def valida_telefono(numero):  
    # Schema: opzionalmente +39, poi 9 o 10 cifre  
    schema = r"^(\+39)?[0-9]{9,10}$"  
    return bool(re.match(schema, numero))  
  
print(valida_telefono("3341234567"))      # True  
print(valida_telefono("+393341234567"))   # True  
print(valida_telefono("abc"))              # False
```